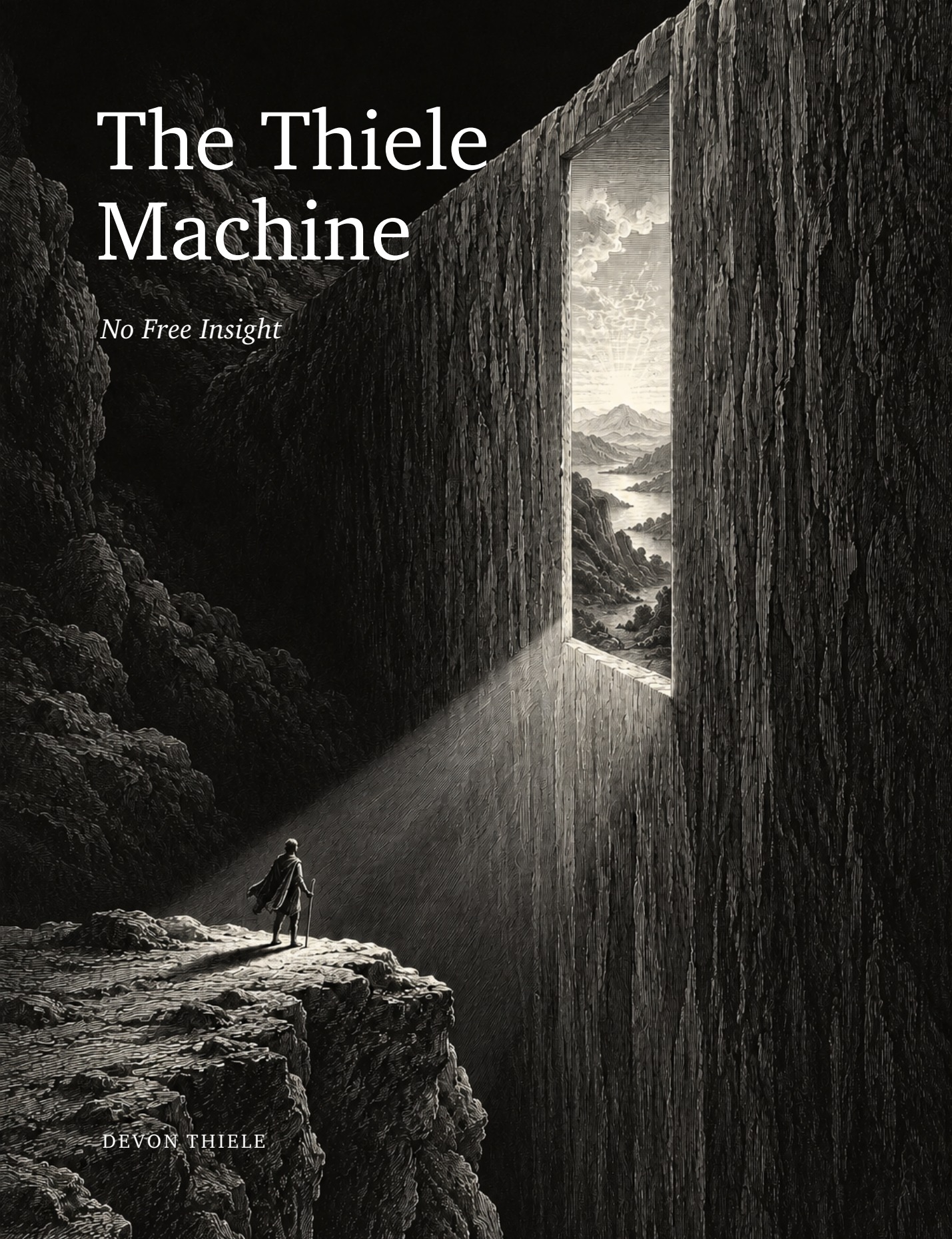
The Thiele Machine

No Free Insight



DEVON THIELE

DEVON THIELE

The Thiele Machine

No Free Insight

The Thiele Machine: No Free Insight

Devon Thiele

Copyright 2025–2026 Devon Thiele. Licensed under Creative Commons Attribution-ShareAlike 4.0 (CC BY-SA 4.0).

The source of this book, its proofs and its build are public: github.com/sethirus/The-Thiele-Machine. Archived at doi.org/10.5281/zenodo.17316437.

Set in Charter. Built from its source with pdfTeX.

The cover and part illustrations were generated with AI in the engraving tradition of Gustave Doré. They are new illustrations, not reproductions of his work.

For Julie, and for my dad.

Julie,

Three years ago, a few months into the rendering engine, I came out of my office and told you I thought I was onto something. You said okay. You didn't ask what it was, or what it was for, or when it would be done, and you didn't ask me for anything I could have shown you, because there was nothing to show. There was a program that had done something it shouldn't have been able to do, and a man who didn't yet know what a Turing machine was.

You gave me the three years anyway. The nights after the lot closed. The weekends. The hours I was in the house and not in the house, because my head was in a wall I couldn't get past. Every dead end, and there were more of those than there are pages here. You never once asked me to stop, and you never once said it was fine when it wasn't. You just kept giving me the next evening.

This book is what I did with them. It's the one thing I've built from nothing that I'm sure of, because I threw out every piece of it I couldn't prove, and what's left holds. I know that isn't what you gave the years for. You gave them for me. But it's what I have to show, and I wanted it to be worth showing.

Dad,

The name on the cover was yours first. I asked you before I used it and you said go ahead. You and Mom taught me that nothing is true just because somebody says it is. This book is that lesson, hundreds of pages long. I put our name on it because I wanted one thing in the world with that name on it that nobody handed me, that I couldn't have faked, and that I can stand behind in front of anyone. Here it is.

Julie carries the name now too. So the machine has it third. This is for the two of you who had it first.

Contents

What this is 8

1 Any machine that steps 11

1.1 Is the pencil thing a computer? 11
1.2 How far does that go? 12
1.3 Try to break it 13
1.4 Now ask it about keeping 14
1.5 The machine that stops 14
1.6 What Turing actually proved 15

2 Running out of room 17

2.1 Four people, three chairs 17
2.2 This is very old 18
2.3 The hotel that never runs out 20
2.4 Cut the strip 21
2.5 How long can a finite machine hold off? 22
2.6 What's being counted 23

3 Why charge for it 24

3.1 A rule you can refuse 24
3.2 The first door: endless paper 25
3.3 The second door: let it come back down 25
3.4 The third door: don't charge for it 26
3.5 How much 27
3.6 Found, assumed, chosen 28

What this is

I think in shapes, so I'll start with one.

Take a thing. Anything. A coin on a table, a light switch, a page in a notebook. Right now it's one way. It could be another.

Now do something to it. Flip the coin, hit the switch, write on the page. That's a move: a direction that takes the thing from how it is to how it is next.

Here's a computer you can run with a pencil. Write 3 on a scrap of paper. The move is: erase the number and write the next one up. Do it twice. 3, 4, 5. You just added 2 and 3. Nothing happened there but a thing, a move, and the next thing. Hold on to that picture; whether it covers every computer is something I'll have to earn, and I will, later, with you watching.

Here's the part I couldn't let go of. Some moves just shuffle. Flip the coin and you can flip it back, and nothing is different for good. But some moves make a difference that stays. Write a word in pen. Blow a fuse. Check an answer and mark it checked. After that move something is the case that wasn't before, and it stays the case.

I figured there had to be a cost in that step. Not in the shuffling. In the step where something goes from not settled to settled and stays there. That was a hunch, so test it on the pencil computer.

Tell it when to stop: count up, and when you reach 5, stop. Ask one question of each number: is it done? 3, no. 4, no. 5, yes. And once it says yes it keeps saying yes, because the machine has stopped.

A machine that has stopped is still somewhere on the next tick. It stays where it is: 5 goes to 5. Draw the arrows. 4 goes to 5. 5 goes to 5. Two different starting places, one landing place. Standing on 5, you can't tell whether you just got there or have been there all along.

Could you dodge that? Try letting the stopped machine rock back and forth instead of sitting still: 5-tick, 5-tock, 5-tick, both of them saying done. Draw the arrows. 4 goes to 5-tick. 5-tock goes to 5-tick. Two arrows land on 5-tick, the spot where you came in. Rocking only moved the place where they meet. The one way out is to never land anywhere twice: make every tick write something new, a tally mark, the time, anything, so the paper never repeats. That works as long as there's paper. On a finite sheet you run out, and two arrows land on one spot again.

So whenever this machine settles its answer and keeps it, something gives. Either two states become one, or it uses up fresh paper on every tick for as long as it runs.

That part isn't a hunch. You drew it. You drew it for one small counter; the book proves it for any machine that goes from one state to a next, and shows you where.

What I do with it is a choice, and I'll say so: I charge for the move where the answer goes from not done to done. Charging for it is up to me. That something has to give once it makes that move isn't. The two arrows don't always meet on that very move. Give done two spots, 5a and 5b: 4 goes to 5a, 5a goes to 5b, and 5b stays put. Now the arrows from 5a and 5b meet on 5b, a tick after the answer settled. But after the move from not done to done, there's no avoiding it: two arrows meet somewhere, or the paper never repeats.

Two more things have to be pinned down before the rule means anything, and you can see both with what's already on the table.

First, take the coin again. Flip it, flip it back, forever, free. Ask whether it's settled. It never is. A rule that says "a move that settles something has to cost" never asks the coin for anything, because the coin never settles anything. So the rule alone would let a coin that does nothing count. The machine has to actually settle something, at least once.

Second, suppose a later move could pay you back. The move that settles charges you one, the next move hands you one back, and the whole run comes out free. So prices can be nothing or something, never a refund.

One last thing before the whole definition. Settled doesn't have to be just yes or no. Take a checklist with three boxes. None ticked, one, two, all three: each tick settles it a little more, and ticks don't come off. "Done or not" is the smallest case, a checklist with one box. So when I say "more settled" from here on, picture the checklist.

So here's the Thiele Machine, all of it. A thing. Moves that take it to the next thing. A way to tell how settled the thing is, where settled never goes back to unsettled. Some run, from a start you pick, that actually settles it more than it was. A price on every move, nothing or more, never a refund. And one rule: a move that settles something more than it was costs something.

That's the whole definition. The rest of the book works out what follows from it, one step at a time, and tries to break it.

Now look back at the pencil computer with a stop in it. That's Turing's machine, in miniature. Turing's machine, from his 1936 paper, is a tape and a table of rules: from where it is, the table says where it goes next. A thing and moves. It already has something that settles and stays settled: whether it has stopped, the same question you asked of 3, 4 and 5. What Turing never put in is a price on the move that stops it. Put one in, and a Turing machine that ever stops fits every line above. So this isn't a rival to Turing's machine, and it computes nothing his can't. His machine is one kind of Thiele machine, with the price left out.

Now try this. Give two people the same messy list of numbers. The first sorts it, then goes back down it checking that every number is no bigger than the next. The second copies a sorted list off somebody else's sheet. Now look at the two sheets. Same numbers, same order. Look as hard as you like: nothing on either sheet tells you which one was checked. The difference was real, and neither sheet kept it. It's the same thing you saw standing on 5: you can see where you are, and not how you got there.

So put a box on each sheet that says checked. A plain Thiele machine charges for ticking the box, but nothing in the rules above stops the copier from paying and ticking it anyway. Paying for the tick doesn't make it true. So add one more condition: the box can only be ticked right after a check of that very list passes. Now the copier's tick is refused, and a ticked box tells you a check of that list passed. I call a box like that established. It's a condition you put on the machine. The bare definition doesn't hand it to you.

It's called the Thiele Machine. The afterword says why.

CHAPTER 1

Any machine that steps

When I started all this I didn't know what a Turing machine was. I knew it had something to do with computers, and that's it. I had a program that had done something I didn't expect, and a hunch, and no words for either. Most of the words in this book I found the same way: I'd hit a wall, go looking for whoever had been there before me, and not move on until I could watch the idea actually work. This chapter is the first of those trips. I'll take you on it the way I took it.

The question I had was the one I left you with on page one. I'd been calling that pencil-and-scrap-of-paper thing a computer. Is it? Or did I just make it small enough to look like one?

1.1 Is the pencil thing a computer?

Start with what the word used to mean. Before about 1940 a computer wasn't a machine. It was a job. In the late 1800s the Harvard Observatory hired rooms full of women to do the arithmetic of the sky, cataloguing stars by hand for a fraction of what a man was paid. Some of them found things nobody else had; Henrietta Leavitt worked out a way to tell how far away a star is. (I got that from Dava Sobel's history of those women and from the historians at the American Institute of Physics.) Rooms like that worked out artillery tables in both world wars; that one's from David Alan Grier's history of human computers.

So the pencil thing wasn't a toy version of a computer. For a long time it was the only kind there was. Good start, but that's history, not proof. A person with a pencil can do a lot of things that aren't following rules. I wanted to know what was left if you took the person away and kept only the rules.

That's exactly the question Alan Turing asked, in a paper in 1936. Going in, all I knew was that it had something to do with computers. What's in there is a clerk. He wrote down, as plainly as he could, everything a person sitting at a desk with a pencil does when they follow a procedure, and threw out everything else. What was left had four parts, and you can build it on the table in front of you right now.

Draw a strip of three squares. That's the tape. Write 1 in the first square, 1 in the second, leave the third blank.

Put a coin on the first square. That's where the clerk is looking. Turing called it the head.

On a scrap of paper write the letter A. That's which step of the procedure the clerk is on. Turing called it the state.

Now the rules. Write these two on a card:

> If you're on A and looking at a 1: leave it, move the coin one square right, stay on A.
>
> If you're on A and looking at a blank: write a 1 there, don't move, and change the letter to STOP.

That's the whole machine: a tape, a head, a state, and a card of rules. Now run it. Each tick, read the card and do what it says. After each tick, write down everything about the table: what's on each square, where the coin is, and the letter. I'll put brackets around the square the coin is on.

[1]	1	␣	A
1	**[1]**	␣	A
1	1	**[␣]**	A
1	1	**[1]**	STOP

It started with two 1s and ended with three. It added one. That's the same thing the pencil computer did when it went from 4 to 5.

Now look at what you wrote down on each line. The whole tape, where the coin is, and the letter. Is there anything about this machine at that moment that isn't on the line? Try to think of something. The rules don't change. The tape, the coin and the letter are all there is. So one line is the whole thing, and the card takes one line to the next line. Thing, move, next thing.

So the pencil computer wasn't a cheat. Turing's machine is the pencil computer with a bigger piece of paper.

1.2 How far does that go?

It would be nice if the answer were "every computer, done." It isn't quite. Push on it and it came apart into three different claims that people tend to run together. Here they are, and what each one stands on.

The first one is the one we just did. Every machine you can write down as a whole state and a rule for the next state has the shape from page one. You checked that yourself for Turing's machine: one line, one card, next line.

The second is that the other ways people have written computation down, with registers instead of a tape, or with functions handed to functions (that's Alonzo Church's lambda calculus, from the same year as Turing's paper), compute exactly the same things Turing's machine does. I didn't prove that, and I won't pretend I did. Other people proved it, one pairing at a time; Turing himself sketched the

match with Church's version in an appendix to his 1936 paper. What you can check yourself is that each of those is also a whole state and a rule for the next one, the same way you just did with the coin.

The third is the big one, and here's where I had to be careful. Does this cover everything a person could ever do by following rules? Does it cover every machine anybody could ever build out of real stuff? Those turned out to be claims nobody can prove, and for a good reason. "Everything a person could do by following rules" isn't a precise enough thing to prove anything about. That claim has a name, the Church-Turing thesis. "Every machine you could build" is a claim about physics, and the only way to knock it down would be to build one that beats Turing's, which nobody has done. Neither is a theorem, and in this book neither will get treated like one.

So the debt from page one comes out like this. For every machine you can write down as a whole state and a rule for the next state, paid: you just checked it. For every machine anybody could ever build, that's a claim about the world, it's labeled as a claim, and I'm not going to lean on it.

1.3 Try to break it

Now the question that actually mattered to me. Page one showed that when the counter settled on 5, two arrows had to meet, unless it used fresh paper forever. That was one little counter. Is it true of every machine with this shape?

Don't take my word for it. Get a sheet of paper and do this.

Draw some dots. Ten, twenty, as many as you like. Those are states. From every dot, draw exactly one arrow to some dot. It can point anywhere, even back at the dot it starts from. That's the one rule: every state has exactly one next state.

Now pick a dot to start on and follow the arrows. Keep a list of every dot you've been on.

Do it a few times, with different drawings and different starts. Watch what happens at each step. You'll find that each step does one of three things.

It lands on a dot that isn't on your list yet. A new one.

It lands on the dot you started from. Now you're going around in a circle, and since every dot has only one arrow out, you'll go around that same circle forever.

Or it lands on some other dot that's already on your list. Look at the first time that happens. The first time you got to that dot, you came in on an arrow from the dot right before it on your list. Now you've come in again, on an arrow from the dot you were just on. Those have to be two different dots, because until this step you hadn't been on any dot twice. Two arrows, from two different dots, landing on one. That's a merge, the same thing as 4 and 5 both going to 5. And from there you're on a path you've walked before, so you walk it again, forever.

Now try to make it do a fourth thing. Really try. Draw whatever you want.

Here's why you can't. Wherever the next arrow points, that dot is either on your list or it isn't. If it isn't, it's new. If it is, it's either the start or it isn't the start. There's no other place for the arrow to point. Three cases, and they cover everything.

And notice what you never had to know: how many dots there were. Ten dots or a billion or an endless sheet, it's the same three cases. What you did need was that every dot has exactly one arrow out. Hold on to that, because it's about to matter.

1.4 Now ask it about keeping

Go back to your dots. Color the dot you start on white. Pick some other dots and color them black, and make one rule about the coloring: no arrow is allowed to go from a black dot to a white one. Once you're on black, you stay on black. That's "settled stays settled." White is not done, black is done.

Follow a run from your white start until it lands on black. Now which of the three things can happen after that?

It can't circle back to the start. The start is white, and you can never get back to white once you're on black.

So it's one of the other two. Either it keeps landing on new dots forever, or two arrows meet somewhere.

Try to draw a counterexample: a run that goes from white to black, stays black, never lands on a new dot forever, and never has two arrows meet. You'll be drawing for a while. There isn't one, and the three cases above are the reason.

That's what page one promised: "the book proves it for any machine that goes from one state to a next." You drew it for one counter there. Now you've drawn it for any machine at all, any number of dots.

What it doesn't tell you is anything about heat. Whether two arrows meeting gives off heat in a real device is a question about the device. People argue about it, and they get their own chapter. This is dots and arrows and nothing else.

1.5 The machine that stops

Here's where the "exactly one arrow out" part bites.

Go back to the tape and the coin. The last line was 1 1 [1], STOP. What does the card say to do next? Nothing. There's no rule for STOP. The machine just stops. In dots and arrows: the STOP dot has no arrow out.

That breaks the one thing the three cases needed. So try the keeping question on a machine that stops. Draw two dots. "Working" is white. "Stopped" is black. One arrow, from working to stopped, and no arrow out of stopped.

Follow it. White, then black, then nowhere. Did two arrows meet? No, there's only one arrow. Did it use up new dots forever? No, it stopped. The answer settled and nothing gave.

Now ask it the obvious question: what does a stopped machine do on the next tick? Not "nothing happens," because something is still there. The usual answer is that it stays where it is. So give stopped an arrow back to itself. Now look again. Working points to stopped. Stopped points to stopped. Two arrows, one dot. The merge is back.

What if you don't want it to stay put? Point stopped at a new dot, "parked," and point parked at itself. Now stopped and parked both point at parked. Two arrows, one dot again. The merge moved one dot over. It didn't go away. That's the rocking machine from page one.

So a machine that stops only dodges this by not answering what happens next. Answer it any way you like, and you're back in the three cases: either two arrows meet, or you draw new dots forever.

1.6 What Turing actually proved

Almost every account you'll find says Turing proved the famous halting problem: that no machine can look at every other machine and say whether it stops. Since the stopping machine is right in the middle of this chapter, I went and looked at what he actually wrote, with an AI walking me through the paper and the links it turned up, and it isn't quite what happened.

Turing's own machines in 1936 weren't built to stop at all. The ones he cared about printed out the digits of a number, forever. What he proved no machine could always decide was a question about those: whether a machine keeps printing digits, and whether it ever prints a given symbol. The second one is a few lines away from the halting problem, so in a real sense he had it. But he never wrote it down that way. Stephen Kleene stated it in 1952. Martin Davis seems to be the first to have called it "the halting problem," in 1958. I found that in a paper where Joel David Hamkins and Theodor Nenu went back to Turing's 1936 paper and checked, line by line.

So the machine that stops, the one sitting right in the hole, is Turing's machine the way people later learned to read it. It's the one page one meant when it said Turing's machine already has something that settles and stays settled: whether it has stopped.

Put it together. Read a Turing machine through "has it stopped?" Give the stopped line a next line by letting it stay put. Then the step that stops it is a step where two arrows meet, which you just drew. That's the step I charge for. Charging for it is still my choice. That something gives there isn't.

The dots in this chapter could go on forever. The next question is what happens when they can't, when the paper runs out.

CHAPTER 2

Running out of room

The last chapter left one door open. A run that settles its answer and keeps it either has two arrows meet somewhere, or it lands on a new dot every step, forever. Endless fresh paper. On a sheet that never ends, that's a real way out.

But nobody has a sheet that never ends. So what happens when the paper is finite?

2.1 Four people, three chairs

Do this one with real chairs, or with coins on a table. It takes about a minute.

Draw a box on the floor and call it the done room. Put three chairs inside it and one chair outside, by the door. Sit four people down, one per chair. The people are the states the machine could be in. The three inside are already done. The one by the door isn't.

One move of the machine moves everybody at once: each person gets up and goes to whatever chair the rule sends them to. There are two rules about where they can go.

Nobody inside may end up outside. Done stays done.

The person by the door has to end up inside. This is the move that settles their answer.

Now try to finish the move with one person in each chair.

You can't. The three inside have to stay inside, so they fill the three inside chairs between them. The person from the door has to come in too, and there's no fourth chair. Somebody has to share. Two people, one chair: two arrows, one landing place.

Try it with more chairs. Ten inside, one by the door. Eleven people have to fit into ten chairs. A thousand inside, one by the door: a thousand and one people, a thousand chairs. It doesn't matter how big the room is, as long as it has a size. The insiders take every inside chair, and the newcomer has nowhere to go but somebody's lap.

Notice who ends up sharing. It doesn't have to be the newcomer. The rule could send the newcomer to an empty-looking chair and send two insiders to the same

chair somewhere else. Page one already showed you that: 4 went to 5a, and the arrows met a tick later on 5b. What's forced is that two arrows meet somewhere in that move. Where they meet depends on the rule.

Two words I'll need from here on. A move is one-to-one when two different starting places always end up in two different places: nobody shares a chair. A move that isn't one-to-one is a merge: two arrows, one landing place.

Now say what the chairs showed in those words. Take any move that keeps done states done and takes one state from not done to done, on a machine with K done states. If the move were one-to-one, the K people already inside would fill all K inside chairs with nobody sharing, and the newcomer would have nowhere to land. So it can't be one-to-one. It merges.

That's the whole argument, and it's the chairs. The only thing finiteness did was make "it fills all K done states" true: K people into K chairs with no sharing leaves no empty chair. That's the one place the size of the room mattered.

2.2 This is very old

The piece of mathematics doing the work there is old, and it isn't mine. Put more things into boxes than there are boxes, and some box gets two. People call it the pigeonhole principle. The earliest use of it I could find is Jean Leurechon in 1622, arguing that somewhere two people must have exactly the same number of hairs on their heads. The first name for it I could find came from Dirichlet, who used it in 1842 and later called it the drawer principle. It's the kind of fact a child accepts and a mathematician spends a whole career finding new uses for.

And the use I just made of it turned out to be old too. I found that out twice, and the second time stung more.

The first time was physics. In 1876 Josef Loschmidt pointed out, against Ludwig Boltzmann's account of how a gas settles down, that the laws of motion run just as well backwards: reverse every molecule and the gas retraces its steps. Twenty years later, in 1896, a young assistant of Max Planck's in Berlin pushed harder. He used Henri Poincaré's recurrence theorem from 1890, which says a system like that comes back as close as you like to where it started, again and again, to argue that nothing in it can climb forever. Give it time and it comes back down. The assistant was Ernst Zermelo, the same Zermelo who would put set theory on firm axioms twelve years later. Boltzmann answered the same year, not very politely, that you'd have to wait so long for a return that nobody would ever see one. (Jos Uffink, who wrote the Stanford Encyclopedia's entry on Boltzmann's physics, calls that reply drenched in sarcasm and bitterness, and calls Zermelo's paper a careful logical argument.)

You can see Zermelo's point on the dots from the last chapter. Draw a finite number of dots where no two arrows ever land on the same dot. Every dot has one

arrow in and one arrow out. Follow any run. Can it ever reach a dot it's been on before, other than the start? No: that would take two arrows into one dot. So since the dots are finite and it can't keep finding new ones, it has to come back to its start. Every run goes around in a loop. Now color the start white and some dots black, done stays done. Can a run go from white to black? If it did, it would have to come back around to its white start, and black can't go back to white. So it can't. A machine where no two arrows ever meet, on a finite sheet, can never settle anything new for good.

That's Zermelo's objection, with the infinities taken out. On a finite set you don't need Poincaré's theorem. And the wait isn't enormous; it's at most as many steps as there are dots. Zermelo was aiming it at the second law of thermodynamics, and settling an answer never came into it. A quantum version came in 1957, when P. Bocchieri and A. Loinger proved that a closed quantum system comes back close to where it started. That's a neighbour, not the same proof: even a small quantum system has endlessly many states, and you'd have to say how a record is read off it.

The second time was computer science, and that's the one that stung. Picture a small machine that reads letters, one move per letter. If every letter just reshuffles its states, no two arrows meeting, people call it a permutation automaton, and the sets of inputs such machines can pick out are called group languages. Jean-Éric Pin's 1992 paper on reversible automata sorts out exactly what those machines can recognise, and in that world the chairs argument is folklore: a machine like that can't recognise "has seen a b at some point," because recognising it would mean a done that stays done. Going quantum doesn't get around it. Alex Brodsky and Nicholas Pippenger showed in 2002 that a quantum version that only measures once, at the end, recognises exactly the same group languages. One that's allowed to measure after every letter, and stop on what it sees, does better, and "has seen a b" is one of the things it picks up. A measurement that can stop the machine is a record that stays. That's the extra thing that lets it in.

Then the closest one, and it's recent. In 2023 Thomas Ouldridge and David Wolpert worked out the physics of a small machine built the way real computers are: the same physical step run over and over. They sorted what such machines recognise into two kinds. If the smallest machine for a job has a letter where two arrows meet, every machine for that job does, and on their derivation running it gives off entropy that no cleverer design can get back. If the smallest machine only reshuffles, they find it can run with no unavoidable entropy at all. One of their worked examples is exactly "has not seen a b," which is "has seen a b" turned inside out. So the counting, and a physical price on it, were in their paper three years before this book. Their price comes out of the physics of the device. Mine, in the next chapter, is a choice I make and say so.

There's a cousin that speaks a different language. Constructor theory is David Deutsch and Chiara Marletto's way of stating physics as which transformations a device that runs in a cycle can and can't bring about, and in it a task is irreversible when it's possible and its reverse isn't. Marletto, Vlatko Vedral and their coauthors in 2022 built an irreversible task out of a machine of quantum bits that swap a little with each other, and ran it with photons. Maria Violaris and Marletto argued the same year that this machine, run in a cycle, can turn a pure quantum bit into a mixed one and never back, even though every piece of it can be reversed, and that this suggests erasing may cost more than the usual bound. The chairs are a small, exact, classical relative of that: a set of states the step can't leave, a task that's possible, and no way back. Theirs is quantum and approximate and aimed at physics. Mine is counted. Deutsch and Marletto's paper on information, from 2015, has another neighbour in its section 8.6: measuring one quantity disturbs another for good, and nothing can undo the disturbance while the result is still being kept. That's a second way of saying that a record that stays makes the step impossible to run backwards.

And Morgan Rogers, in his papers of 2019 and 2021 on what mathematicians call toposes of monoid actions, gives the general home for one more way to see it: if every move can be undone by some other sequence of moves, you can never enter a room you're not allowed to leave, because the undoing sequence would have to take you back out. On a finite set, a move where no two arrows meet can always be undone that way: repeat it enough times and everybody comes home.

So the counting isn't mine. It had homes in physics and in automata long before me, and a physical price on it a few years before me. What I do with it starts in the next chapter.

2.3 The hotel that never runs out

Now the way out. If the chairs run out, give the room more chairs. Endlessly many.

Take a strip of paper that never ends. Each move writes a 1 in the next empty box. Done means "there's at least one 1 on the strip." Check it against everything so far. Done stays done: once there's a 1, there's always a 1. The first move takes the empty strip to done. And no two arrows ever meet: two different strips always become two different strips, because you can always erase the last 1 and get back where you were. So here's a machine that settles something, keeps it, and never merges anything.

It got out of the chairs. It didn't get out of paying. Look at what every move does: it writes in a box nobody has used before. The first move does it, and so does every move after that, forever. That's the other column from the last chapter: fresh paper on every tick.

If the endless strip feels like a cheat, it's a famous one. David Hilbert told a story in a lecture course in the winter of 1924 to 1925 about a hotel with endlessly many rooms, every one of them taken, where a new guest still gets a room. Everybody moves from room n to room n + 1, and room 1 is left empty. Nobody shares. The one-to-one move leaves a gap. That's the strip.

The part worth checking is that Hilbert never published it. The hotel got famous because George Gamow put it in a popular book in 1947, and for a long time it wasn't clear Hilbert had told it at all. The historian of science Helge Kragh first guessed that Gamow made it up. Then Tilman Sauer, who had helped edit a volume of Hilbert's lectures, pointed him to Hilbert's lecture notes, which had just been printed in 2013, and in 2014 Kragh took his guess back in a paper called "The True (?) Story of Hilbert's Infinite Hotel." Keep the question mark. Even Kragh's paper puts the lecture in January 1924 in one place and January 1925 in another. Still, this time a story everybody repeated had a real lecture behind it. I've learned not to count on that.

Bring it back to the chairs. In the closed room the person from the door had nowhere to sit because the chairs ran out. In Hilbert's hotel they never do. So the whole argument was about running out of room, all along.

You can run the chairs backwards to see exactly how much room. Take a move that keeps done states done, settles one new state, and never lands two arrows on one place. Everybody already in the done room moves to a different done chair. The newcomer gets a done chair too, and nobody already inside is moving into it, because that would be two arrows meeting. So everybody already inside got seated in the done room with at least one chair to spare. With n chairs, that's n people into n - 1 chairs, which can't be done. So a move like that needs a done room with no end to it. Give the room any fixed size, and the move merges.

That's Hilbert's front desk run backwards. It's also exactly how Richard Dedekind defined "infinite" in 1888: a collection is infinite when you can match it one-to-one with a part of itself that leaves something out. So taking fresh paper isn't a way out of paying. It's the other way to pay, and only a machine with endlessly many states can keep paying it.

2.4 Cut the strip

Now cut the strip to eight boxes. Same rule: fill the next empty box from the left, never touch the earlier marks. There are nine possible strips, from empty to full. Write them down:

0 marks	`00000000`	next: `10000000`
1 mark	`10000000`	next: `11000000`
⋮	⋮	⋮
7 marks	`11111110`	next: `11111111`
8 marks	`11111111`	next: ?

After eight moves the strip is full. What should the ninth move do? Try each answer.

It stays full: then the strip with seven marks and the full strip both go to the full strip. Two arrows, one landing. The chairs.

It stops: then it has no next state, and that's the stopping hole from the last chapter.

It clears the strip: then done went back to not done.

Every answer gives up one of the things the chairs needed. None of them gets past the chairs.

Keeping the whole history like this is the trick Charles Bennett used in 1973, when he showed that any computation can be done without two arrows ever meeting, if you let it keep a copy of everything it did. Rolf Landauer, back in 1961, had already written that for a machine that runs a program that doesn't stop, the space to hold every step "cannot be there." Keeping the history is a real way to pay. It costs a memory that never fills, and every machine you can buy has a memory that fills.

Bennett wasn't the first there, or the last. Yves Lecerf showed in a short note to the French Academy of Sciences in 1963 that machines with no merging can imitate any Turing machine, and hardly anybody noticed. Tommaso Toffoli in 1980, and Edward Fredkin and Toffoli together in 1982, both then at MIT, built logic gates and circuits that compute anything and lose nothing, by carrying extra inputs in and extra outputs out. Their basic gate wasn't new even then: Landauer's own 1961 paper writes one down, calls it reversible and able to do an AND, and doubts anything like it could be useful. What Toffoli added was the proof that it computes everything. And Kenichi Morita proved in 1996 that even a tiny machine with two counters can compute everything with no two arrows ever meeting. Every one of them keeps what it doesn't forget somewhere: a history, extra outputs, a counter that codes the past. They all pay in fresh paper.

2.5 How long can a finite machine hold off?

One more question, because the 5a and 5b example on page one hinted at it. If the machine has a few spare done states, can it put off the merge for a while? Yes, and you can count exactly how long.

At three o'clock a screen says 42. The program is finished. Leave the camera running. At 3:01 it says 42. At 3:02 it says 42. The screen hasn't changed, but

the box behind it still has a next state every tick. It can stay exactly where it is, or something inside it can move while the screen stays put: picture a wheel turning from "first holding position" to "second holding position."

Say the box has K different ways of holding the answer 42. Start one tick before it finishes, and see how many ticks it can go without two arrows meeting. Can two of the holding ticks be the exact same state? If they were, follow the arrows backwards from both, one tick at a time. Since no two arrows meet, being on the same state now means they were on the same state a tick ago. Keep walking both back. Eventually the earlier one reaches the tick before it finished, the state that wasn't done yet, while the later one is still on a done state. A state that isn't done would have to be the same state as one that is: impossible. So every holding tick needs its own state, and with K of them, it can last at most K ticks. Within K + 1 ticks of finishing, two arrows meet.

You can build one that lasts exactly that long: the unfinished state, then K holding states in a row, the last one stepping to itself. The answer holds for K ticks with nothing merging, and on the next tick the last two arrows meet. A bigger memory puts off the merge. It doesn't make it go away.

2.6 What's being counted

One thing to keep straight before going on. Whether a move merges depends on what you count as the state. The endless strip keeps its whole history in the state, so nothing ever merges; cut it to eight boxes and the same rule merges. Keep a list of every earlier state alongside the current one, and any machine's moves become one-to-one, because two different histories never look the same; a later chapter proves that. Keep only the state one step back, and it isn't enough: two states that differ only in what came before them still land together.

So "this move merges" is always a fact about a move on a stated set of states. Every merge in this book is counted on the whole state the machine says it has, and nothing outside that state is counted.

What I've got now is counting and nothing else: on a finite machine, settling something for good means two arrows meet. The next chapter is about what I do with that, and which parts of it are my choice.

CHAPTER 3

Why charge for it

Everything so far has been counting, and the counting isn't mine. On a finite machine, settling something for good means two arrows meet somewhere. On an endless one, it can mean fresh paper forever instead. Nobody gets to choose that. You drew it.

Page one also said something I do get to choose: I charge for the move where the answer goes from not done to done. This chapter is about how much of a choice that is. If it's nothing but my preference, then the whole machine is a long way of telling you my favourite colour, and you should treat it that way. So I want to find out what it rests on.

3.1 A rule you can refuse

I'm not going to ask you to believe a price. I'm going to write one down as a rule, in the smallest form I can, and you can accept it or refuse it.

Any move where two arrows meet costs at least one.

That's all it says. It doesn't mention done or not done. It doesn't mention checking or settling. It puts a price on one thing: a move that sends two different states to the same place.

It isn't my rule. Charles Bennett, who spent decades working out what computing has to cost, put it this way in 2003: what has to cost is "the erasure of a bit or the merging of two computation paths." Two arrows meeting is two paths merging. I'm taking his rule as it stands and writing it on the move.

Now put it next to the chairs. The chairs said: on a finite machine, a move that settles something for good is a move where two arrows meet. The rule says: a move where two arrows meet costs at least one. Put those two sentences side by side and read them together. A move that settles something for good costs at least one.

That's the rule from page one, the one I said was my choice. I call it the toll. That was my word for it before I could prove any of this: there had to be a toll, a cost in the transition, in the step itself. And look at where it came from just now. Nobody handed it to the machine. It fell out of the chairs and Bennett's rule together.

So it isn't only my preference. But it isn't free of choices either. Count what went into it. There are three, and you can refuse each one.

The done states have to be finitely many, so the chairs run out.

Done has to stay done.

And two arrows meeting has to cost something.

Accept all three and you don't get to refuse the toll. It comes with them. Refuse any one, and the toll goes back to being my choice. So the question "did I have to pick it?" has turned into three smaller questions, and those are better things to argue about. For each one there's a machine that walks out through that door, and you should see each one, because a rule you can't picture breaking is a rule you're taking on faith.

3.2 The first door: endless paper

You've already seen this machine. It's the strip from the last chapter that never runs out. Each move writes a 1 in the next empty box. Done stays done, the first move settles it, and no two arrows ever meet.

So Bennett's rule never charges it anything. Two arrows never meet, so there's nothing to charge. The strip settles its answer at no cost on the price list.

But it didn't get out of paying. It pays in fresh paper, on every move, forever. A price list that only charges for arrows meeting just isn't looking at that column. And on any real machine, the paper runs out, and the chairs come back.

3.3 The second door: let it come back down

Take one coin. The only move is: turn it over. Heads is done, tails is not done.

Draw the arrows. Tails goes to heads. Heads goes to tails. Two dots, two arrows, and they never meet. So Bennett's rule never charges it.

And it does settle something: tails goes to heads, not done to done. At no cost.

What it broke is the second condition. The same move that took tails to heads takes heads back to tails. Done doesn't stay done. So it's not keeping anything. It's flipping a switch back and forth, and the page-one definition already throws it out: a Thiele machine's done never goes back.

Here's the part of this I found sharpest. Forget about done staying done entirely, and keep only the finite chairs. Take any move that takes some state from not done to done. Now go through the done states, one at a time, and ask: does this move send any of them back to not done?

If it does, you've found the coin: the move takes something back somewhere.

If it doesn't, then this move keeps every done state done. That's all the chairs needed, for this one move. So two arrows meet.

There's no third answer. On a finite machine, a move that settles something either has two arrows meet, or it takes something back somewhere else. A move that settles for free and never takes anything back doesn't exist.

That's narrower than "every record costs." It's still true, and I'll take it.

One more picture, so you don't think it's tidier than it is. Three cards, T0, T1 and T2, with a star on T1. The move sends T0 to T1, T1 to T2, and T2 to T1. Look at what it does. T0 gains the star: something settled. T1 loses the star: something taken back. And T0 and T2 both land on T1: two arrows meet. So Bennett's rule charges this move, and yet it doesn't keep anything for good. Charged moves and permanent moves aren't the same list. What's true is one direction: on a finite machine, every move that settles something for good is a move Bennett's rule charges.

And there's an exact way to say which moves Bennett's rule charges no matter what the rest of the price list looks like. A move where two arrows meet gets charged by every price list that follows the rule; that's what the rule says. A move where they never meet doesn't have to be: price that one move at zero and every other move at one, and the rule is still followed. So the moves the rule forces you to charge are exactly the moves where two arrows meet. That one holds on any machine, finite or not.

3.4 The third door: don't charge for it

Take a sheet of paper. It's either blank or stamped. One move: stamp it. Blank goes to stamped, and stamped stays stamped.

Check the conditions. Two states, so the chairs run out. Stamped stays stamped. Blank goes to stamped, so something settles. And draw the arrows: blank goes to stamped, stamped goes to stamped. Two arrows meet. Everything the chairs needed is here.

Now set the price of the stamp to zero.

Nothing in counting stops you. The stamp merges two states, and you just decided merges are free. The answer settled, and the bill says zero.

Price the stamp at one instead, and every condition holds at once, so the three can live together. That stamp at one is the smallest Thiele machine there is, and it gets its own chapter. But the third door asks whether they have to live together, and counting can't say so. The only thing that could close this door is a claim about the world: that two arrows meeting really does cost something, out there, in a real device.

The nearest such claim is Rolf Landauer's, from 1961: that forgetting gives off heat. It's measured for a few devices and argued over for everything else, and it gets its own chapter, with the people who doubt it named. I'll say one thing about it now, because it matters here. Bennett's rule as I wrote it charges every move where two

arrows meet, whether or not you already knew which state you were in. Landauer's heat, as that chapter will show, is only owed for what you didn't know. The rule I'm using is the worst case on purpose: a move that has to work whatever it's handed pays for the worst it could do.

So that's where the toll stands. Accept finite chairs, done stays done, and a price on two arrows meeting, and the toll isn't a choice; it's what the three say together. Each of the three is a choice, and I've shown you a machine walking out through each door. The physics that would close the third door is a premise, and I'll always call it one.

3.5 How much

The toll says at least one. Counting can say more, if you let it.

Take four cards and write A, B, C and DONE on them. Pick one without showing me. Now cover its face with a label that says DONE. Whatever you picked, the face now says DONE. Before the label, how many yes-or-no questions would it take me to find out which card you had? Two: "Is it A or B?" and then one more. After the label, I don't need to ask anything, because there's only one thing the face can say. Two questions' worth of difference got squeezed out.

That's four states landing on one. Each question you no longer need halves the number of places you could have been: four to two, two to one. Two halvings.

If you're willing to go one step further than Bennett's rule, and say each unit of cost pays for at most one halving, then that move costs at least two. And two is enough. In general: if m states were already done and the move settles k more, all m + k of them land on the m done ones, and the move has to pay at least as many halvings as it takes to get from m + k down to m. One newcomer next to one done state is one halving: the toll again.

(The label can be peeled off, so the cards only show the squeeze on what the face says. Whether anything physical got erased is a different question, for the heat chapter.)

There's a plainer way to say what the halving rule asks for. Look at one move and find the place where the most arrows land. Call that its worst pile-up. A cost of one allows piles of two. A cost of two allows piles of four. A pile of three already needs two. That's why the m + k states in the done room had to pay at least that much: spread m + k arrows across m chairs as evenly as you like, and the fullest chair still gets at least (m + k) / m of them. So the rule asks every move to pay for the worst it could ever do, whichever state it's actually handed.

That worst case has a home in physics. Philippe Faist, Frédéric Dupuis, Jonathan Oppenheim and Renato Renner worked out in 2015 the least work any logical step needs when it has to succeed every time, on every input, with no averaging. It

comes to exactly that logarithm of the worst pile-up. An AND gate sends three of its four inputs to 0, so it needs about 1.6 halvings' worth, two in whole units. My rule counts the same thing in units of cost. Joules don't come into it.

You can also count the whole run this way. Write N different starting states on cards. Run each card through the rule once, and stack the cards that land on the same state into one pile. Do it again with the piles. Each tick, count how many different states you're down to. The drops add up exactly: what you started with equals everything lost along the way plus what's left at the end. If you end up with fewer than you started with, some tick had two arrows meet. Turn the cards over first, though. Two runs can show the same answer on the front and still be different underneath, and those are different states.

A word about the k in all this. Physicists write kT for heat at a temperature, and when the counting meets the physics in the heat chapter you'll see k with a little B hung under it, for Boltzmann. That letter has a story I didn't expect. Boltzmann's grave in Vienna has $S = k \log W$ carved on it: entropy is a constant times the logarithm of the number of ways the parts could be arranged. The idea is his. But the constant, and the equation written that way, came from Max Planck, around 1900. In his Nobel lecture in 1920 Planck said the constant "is frequently termed Boltzmann's constant, although to the best of my knowledge Boltzmann himself never introduced it." So the most famous constant in this corner of physics wears the name of a man who never wrote it down. Take the constant away and what's left is the logarithm of a count, which is a number of halvings, which is what the cards just gave you. The constant is the exchange rate between halvings and heat. Whether the world honours that rate is the premise.

3.6 Found, assumed, chosen

That's the place to sort everything so far into three piles, because the rest of the book depends on where each thing falls.

I see it like this: I didn't invent a machine. I found one. That walks straight into one of the oldest fights in mathematics, so here's a short trip into it. G. H. Hardy, the Cambridge number theorist who wrote A Mathematician's Apology in 1940, came down on the found side about as hard as anyone ever has: he wrote that mathematical reality lies outside the people who study it, and the theorems they like to call their creations are really notes on what they saw. The German mathematician Leopold Kronecker supposedly split the difference: God made the whole numbers, and everything else is the work of people. Supposedly, because push on that line and it gets thin fast. It comes down through Heinrich Weber, in a memorial piece written after Kronecker died, pointing back to a lecture from 1886, and that's the only source I could find anybody citing. It's a great line with one witness.

None of that settles whether I found the Thiele Machine or made it up, and a proof wouldn't settle it either. So it's a conviction, and I'll call it one. What it changes is how I hold the thing: like something that was already there, which means I can be wrong about its shape and the shape won't care. And "found" only covers what counting forced. Here are the three piles.

Found: what counting forces on any machine.

Every step of a machine with one next state for each state lands somewhere new, circles back to its start, or has two arrows meet, and after it comes back it repeats forever. You drew that in chapter one.

A run that settles something and keeps it can't circle back to its start, so it has two arrows meet or takes fresh states forever. When only finitely many states are done, the move that settles it has two arrows meet. You sat people in chairs for that.

A machine that stops, with no next state, slips past all of it, and gets caught the moment you say what it does on the next tick.

Assumed: what the found pile is taken to stand on in the world.

That two arrows meeting in a real device, when the device really could have been in either state, gives off heat. That's Landauer's principle, and it's an assumption here, never proved, with its critics named in its chapter.

That a real device can only tell apart finitely many states in any bounded space, so its fresh paper runs out. Nobody has measured where that ceiling is.

That the machine starts clean: if done means "this happened," that's only because it started at not done. That start is put in by hand.

Chosen: what I add, each of which could have gone another way.

To put the price inside the step, as part of what a step is.

That done stays done. That's in the definition because I put it there.

The toll: at least one on the move that settles something. With finite chairs and done staying done, it follows from a price on two arrows meeting, and that price is a choice too.

Which thing gets to count as settling, and what the price list charges. Nothing I've shown you forces either one.

Everything after this chapter is built on those three piles, and I'll keep saying which pile each piece comes from.